

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL 08 – Asynchronous JavaScript (Fetch API,
JSON, REST API Sederhana)**

Nama Mahasiswa : Saprina Melita Sari
NIM : 23051430012
Kelas : A1
Tanggal Praktikum : 17 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

MODUL 8

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 08
Topik Utama	Form Validation & Local Storage
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome	Versi terbaru	Testing & Debugging
3	Live Server	Versi 5.7.9	Extensions VS Code untuk menjalankan html

4	Prettier	Versi 12.3.0	Extensioins VS Code untuk merapikan indentansi code
5	Bootstrap	Versi 5.3.8	Framework front-end open-source untuk mempercepat pengembangan website

D. DASAR TEORI

D.1 Konsep Utama

Modul ini berfokus pada penerapan Asynchronous JavaScript dan penggunaan Fetch API sebagai dasar komunikasi data antara sisi klien (client-side) dengan server eksternal (server-side). Secara konsep, pemrograman asinkron memungkinkan aplikasi web melakukan permintaan dan pemrosesan data di latar belakang tanpa mengganggu aktivitas pengguna pada antarmuka utama. Pada praktikum ini, konsep tersebut diterapkan melalui pengambilan data laporan dari REST API publik. Proses tersebut juga dilengkapi dengan indikator pemuatan (loading) sebagai bentuk umpan balik kepada pengguna selama pertukaran data melalui jaringan sedang berlangsung.

Selain itu, modul ini menekankan pentingnya pemanfaatan JavaScript Object Notation (JSON) sebagai standar pertukaran data modern beserta teknik pengolahannya. Data mentah yang diterima dari server harus melalui proses parsing JSON agar dapat diubah menjadi objek yang mudah dibaca dan diolah oleh sistem. Setelah berhasil diekstraksi, data diproses lebih lanjut melalui teknik mapping dan lokalisasi, seperti membatasi jumlah data yang ditampilkan serta mengganti sebagian isi dengan simulasi teks berbahasa Indonesia. Penerapan tersebut menunjukkan bahwa data dari sumber eksternal dapat disesuaikan dengan kebutuhan dan konteks operasional aplikasi yang dikembangkan.

Pada tahap akhir, modul mengombinasikan manipulasi Document Object Model (DOM) secara dinamis dengan penerapan mekanisme penanganan kesalahan (error handling) untuk meningkatkan ketahanan sistem. Data yang telah distandardisasi kemudian ditampilkan secara otomatis dalam bentuk kartu informasi, di mana setiap kartu memiliki identitas unik (ID) guna mendukung interaksi yang lebih akurat. Sebagai upaya menjaga stabilitas aplikasi, dilakukan pemeriksaan terhadap status respons serta penggunaan blok catch untuk menangani kemungkinan terjadinya kesalahan. Dengan demikian, ketika koneksi mengalami gangguan atau permintaan gagal diproses, aplikasi tetap dapat berjalan dengan baik dan memberikan pesan peringatan yang informatif kepada pengguna tanpa mengalami crash.

D.2 Keterkaitan dengan Teknik Industri

Praktikum pada modul ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, terutama dalam pengembangan Sistem Informasi Manajemen (SIM) dan pemantauan proses produksi secara real-time. Dalam lingkungan manufaktur, informasi terkait kondisi operasional di rantai produksi perlu diperoleh dengan cepat tanpa menghambat aktivitas lain yang sedang berlangsung. Penerapan Asynchronous JavaScript dan Fetch API pada program ini menggambarkan cara kerja dasbor pemantauan industri, seperti Andon system, yang mampu mengambil data berupa status mesin, peringatan kerusakan, atau anomali sensor dari server pusat maupun sistem Enterprise Resource Planning (ERP) secara otomatis di latar belakang. Kemampuan memperbarui tampilan antarmuka tanpa menyebabkan lag atau pembekuan layar sangat membantu operator dan manajer dalam melakukan pengawasan produksi secara responsif dan berkesinambungan.

Hubungan dengan keilmuan Teknik Industri juga tampak pada proses pengolahan dan standardisasi data operasional. Pada era Industrial Internet of Things (IIoT), berbagai mesin, sensor, dan perangkat lunak saling bertukar data dalam jumlah besar. Tahapan pengambilan data melalui REST API, penguraian format JSON, hingga penyesuaian isi data menjadi laporan insiden yang terstandar mencerminkan prinsip interoperabilitas dalam sistem terintegrasi. Proses tersebut menunjukkan bagaimana data dari sistem eksternal atau vendor dapat diterjemahkan ke dalam format yang sesuai dengan kebutuhan operasional perusahaan. Kemampuan melakukan standardisasi data seperti ini menjadi kompetensi penting bagi insinyur industri dalam menyediakan informasi yang valid sebagai dasar evaluasi kinerja mesin dan pengambilan keputusan.

Selain itu, manipulasi elemen DOM yang diterapkan dalam program merepresentasikan alur kerja pada Computerized Maintenance Management System (CMMS) yang efektif dan minim kesalahan. Ketika tombol "Tindak Lanjut" pada kartu insiden dipilih, sistem secara otomatis memberikan umpan balik visual sekaligus menonaktifkan aksi lanjutan pada elemen tersebut. Mekanisme ini sejalan dengan konsep Poka-Yoke, yaitu pendekatan pencegahan kesalahan untuk menghindari pengiriman tiket perbaikan yang berulang atau terjadinya tumpang tindih tugas pada tim pemeliharaan. Didukung dengan penerapan error handling untuk mengantisipasi kegagalan jaringan, rancangan aplikasi ini mendukung terciptanya sistem kerja yang andal, terkoordinasi, mengurangi pemborosan waktu tunggu (waste of waiting), serta selaras dengan prinsip-prinsip Teknik Industri dalam meningkatkan efisiensi operasional.

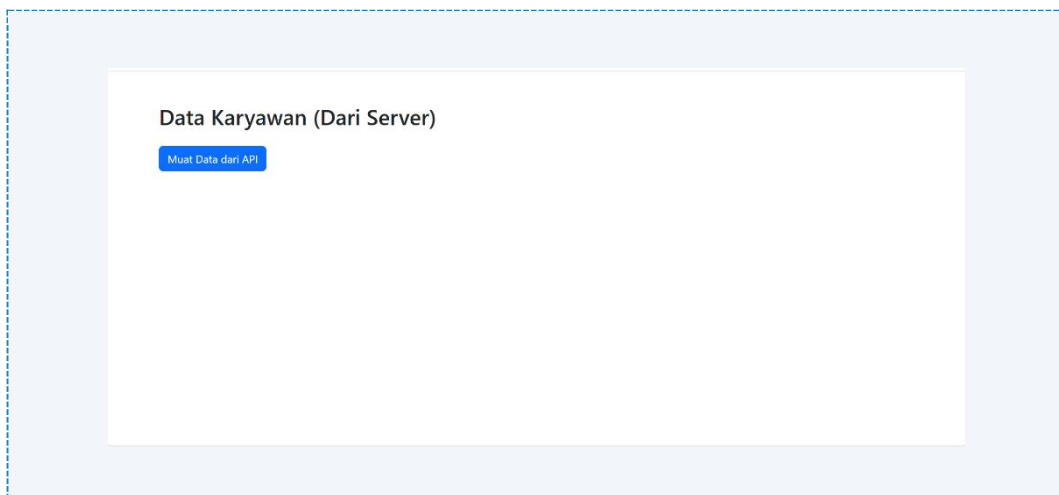
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat HTML untuk Menampilkan Data

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>
  <div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
  </div>
  <script src="api_script.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Fetch Data dengan .then() (Promise)

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error (misal putus internet)
      container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
    })
    .finally(function () {
      // Sembunyikan loading (baik sukses maupun gagal)
      loading.classList.add('d-none');
    });
});

function renderData(data) {
  data.forEach(function (karyawan) {
    // Buat elemen card Bootstrap
    const col = document.createElement('div');
    col.className = 'col-md-4 mb-3';

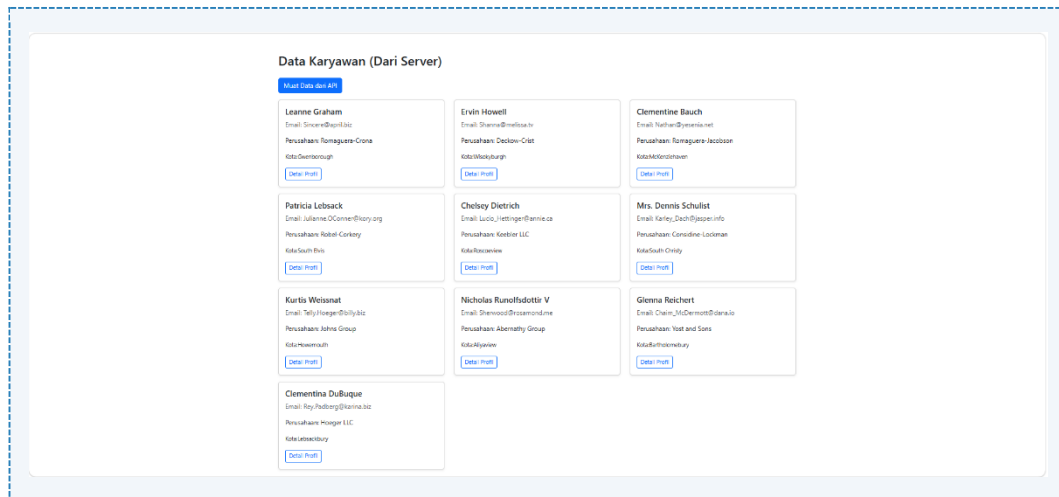
    col.innerHTML = `
<div class="card h-100 shadow-sm">
<div class="card-body">
```

```

<h5 class="card-title">${karyawan.name}</h5>
<p class="card-text text-muted">Email: ${karyawan.email}</p>
<p class="card-text">Perusahaan: ${karyawan.company.name}</p>
<p class="card-
text"><small>Kota: ${karyawan.address.city}</small></p>
<a href="" class="btn btn-sm btn-outline-primary">Detail Profil</a>
</div>
</div>
`;
container.appendChild(col);
});
}

```

Screenshot Hasil:



Gambar E.2 – hasil fetch then()(Promise)

E.3 Langkah 3 – Fetch dengan Async/Await (Modern Style)

```

// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);

    const response = await
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

    if (!response.ok) {
      throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
  }
}

```

```

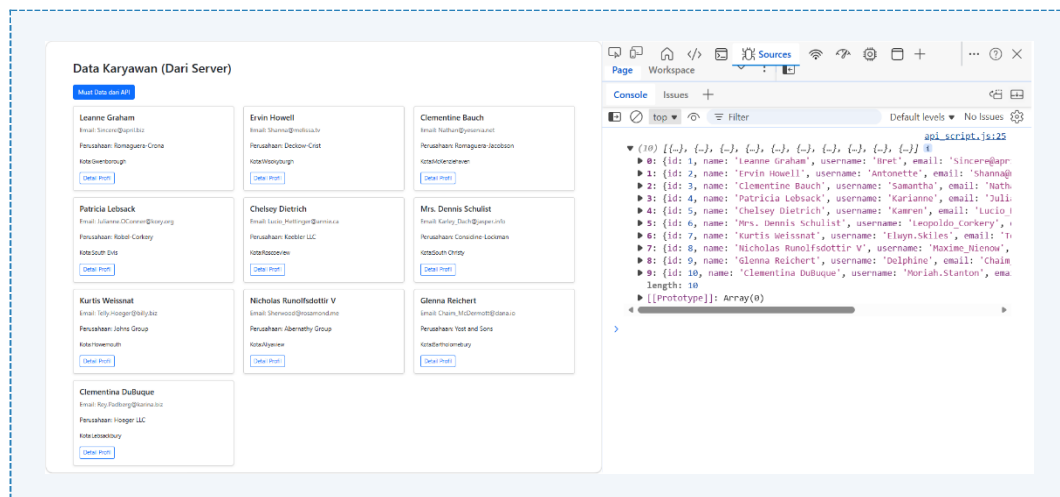
console.log("Ditemukan:", data);
alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);

} catch (error) {
  console.error(error);
  alert(error.message);
}
}

// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.

```

Screenshot Hasil:



Gambar E.3 – Fetch dengan Async/Await

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API.

Jawaban / Kode Solusi:

HTML

```
<button id="btnTambah" class="btn btn-success">Tambah Karyawan Baru</button>
```

JAVASCRIPT

```
document.addEventListener('DOMContentLoaded', function () {

    const API_URL = 'https://jsonplaceholder.typicode.com/users';
    const btnTambah = document.getElementById('btnTambah');

    if (!btnTambah) {
        console.error("Tombol tidak ditemukan!");
        return;
    }

    btnTambah.addEventListener('click', function () {

        console.log("Tombol diklik");

        // DATA LEBIH DARI SATU
        const dataKaryawan = [
            {
                name: "Budi Santoso",
                username: "budi123",
                email: "budi@email.com",
                address: { city: "Surabaya" }
            },
            {
                name: "Siti Rahma",
                username: "siti123",
                email: "siti@email.com",
                address: { city: "Semarang" }
            },
            {
                name: "Andi Wijaya",
                username: "andi123",
                email: "andi@email.com",
                address: { city: "Solo" }
            }
        ];

        // LOOP POST SATU-SATU
        dataKaryawan.forEach((karyawan) => {
            fetch(API_URL, {
                method: 'POST',
                headers: {
```

```
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(karyawan)
    })
    .then(response => response.json())
    .then(data => {
      console.log("BERHASIL POST:", data);
    })
    .catch(error => {
      console.error("ERROR:", error);
    });
  });
});

CSS
/* Background halaman */
body {
  background: linear-gradient(to right, #eef2f3, #dfe9f3);
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

/* Judul */
h2 {
  font-weight: bold;
  color: #333;
}

/* Tombol */
#btnTambah {
  transition: all 0.3s ease;
}

#btnTambah:hover {
  background-color: #198754;
  transform: scale(1.05);
}

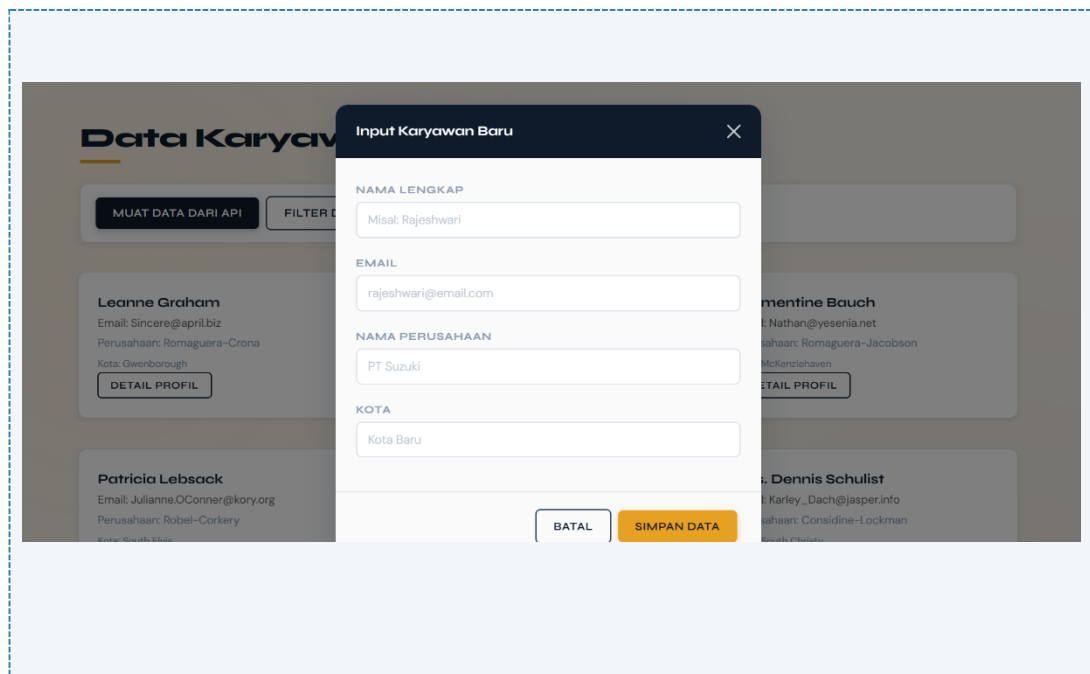
/* Card karyawan */
.card {
  border: none;
  border-radius: 12px;
  transition: all 0.3s ease;
}
```

```
/* Hover card */
.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);
}

/* Isi card */
.card-title {
  font-weight: bold;
  color: #0d6efd;
}

.card-text {
  font-size: 14px;
}

/* Spinner loading */
#loading {
  margin-left: 10px;
}
```



Gambar F.1 – Hasil pembuatan form dan button tambah karyawan

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

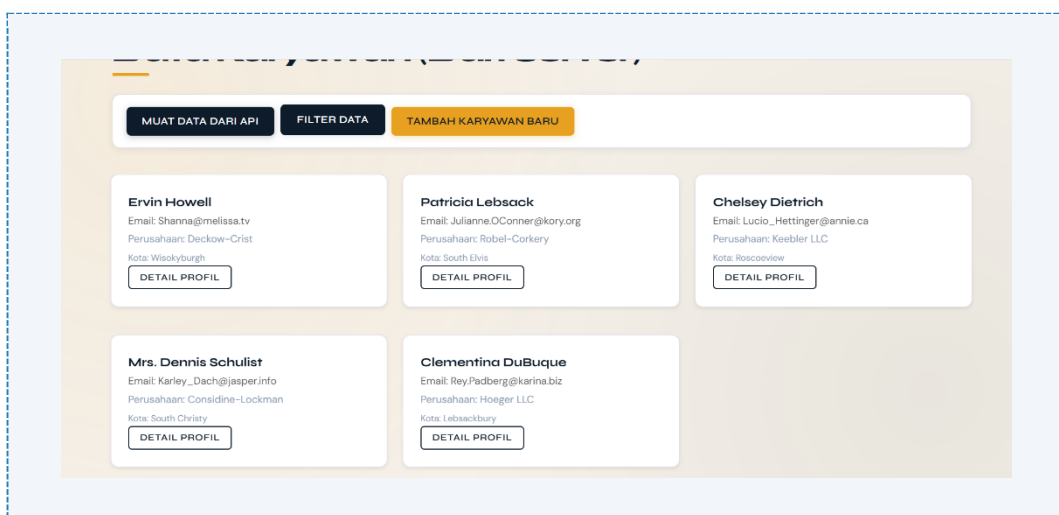
Jawaban / Kode Solusi:

```
const btnSortJumlah = document.getElementById('btnSortJumlah');

<button id="btnFilter" class="btn btn-secondary">Filter data</button>
btnFilter.addEventListener('click', function () {
  // Validasi jika data belum dimuat
  if (daftarKaryawan.length === 0) {
    alert("Harap klik 'Muat Semua Data' terlebih dahulu!");
    return;
  }

  // Filter dari variabel daftarKaryawan
  const hasilFilter = daftarKaryawan.filter(function (item) {
    return item.address.city.toLowerCase().includes('s');
  });

  // Tampilkan hasil filter ke layar
  container.innerHTML = "";
  renderData(hasilFilter);
});
```



Gambar F.2 – Hasil pembuatan button filter data

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

Anggap API `https://jsonplaceholder.typicode.com/posts` Adalah database "Laporan insiden".

- a. Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
- b. Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance"

Penjelasan pendekatan/solusi yang digunakan:

Proses pengerjaan diawali dengan menyiapkan sepuluh data simulasi berbahasa Indonesia yang disesuaikan dengan kebutuhan pelaporan insiden di lingkungan kerja. Setelah itu, sistem diprogram untuk mengambil data melalui API ketika tombol dijalankan oleh pengguna. Selama proses pengambilan data berlangsung, ditampilkan indikator loading sebagai informasi bahwa sistem sedang memproses permintaan. Selain itu, mekanisme penanganan kesalahan juga diterapkan untuk mengantisipasi kemungkinan terjadinya gangguan koneksi.

Setelah data berhasil diperoleh, program secara otomatis membangun tampilan antarmuka dengan menampilkan kartu-kartu laporan pada halaman web. Setiap kartu beserta tombol yang terdapat di dalamnya diberikan identitas unik (ID) untuk memudahkan proses identifikasi dan pengelolaan elemen secara tepat. Pada tahap akhir, tombol "Tindak Lanjut" diprogram agar dapat memberikan respons interaktif. Ketika tombol diklik, sistem akan menampilkan pesan alert yang berisi informasi mengenai status tiket. Setelah pesan tersebut ditutup, kartu dan tombol akan mengalami perubahan tampilan sebagai indikator visual bahwa insiden telah diproses, sekaligus menonaktifkan tombol tersebut guna mencegah terjadinya pemrosesan berulang pada laporan yang sama.

Kode Utama:

```
const API_POSTS = "https://jsonplaceholder.typicode.com/posts";
const API_USERS = "https://jsonplaceholder.typicode.com/users";

let semuaKaryawan = [];

// Ambil elemen
const btnLoad = document.getElementById("btnLoad");
const btnFilter = document.getElementById("btnFilter");
const loading = document.getElementById("loading");
const cInsiden = document.getElementById("containerInsiden");
const cKaryawan = document.getElementById("containerKaryawan");
const statusBadge = document.getElementById("statusBadge");
```

```
const modalDetailBS = new
bootstrap.Modal(document.getElementById("modalDetail"));

// ===== UTILITY =====
const setLoading = (v) => {
  loading.classList.toggle("d-none", !v);
  btnLoad.disabled = v;
};

const inisial = (nama) =>
  nama.split(" ").slice(0, 2).map(w => w[0].toUpperCase()).join("");

const getStatus = (id) => {
  const list = [
    { label: "Kritis", cls: "badge-kritis" },
    { label: "Sedang", cls: "badge-sedang" },
    { label: "Rendah", cls: "badge-rendah" },
  ];
  return list[id % list.length];
};

// ===== INSIDEN (BAHASA INDONESIA)
=====
function renderInsiden(posts) {
  cInsiden.innerHTML = "";

  const data = posts.slice(0, 10);
  document.getElementById("jumlahInsiden").textContent =
    `${data.length} insiden`;

  const daftarMasalah = [
    "Kerusakan mesin produksi",
    "Keterlambatan proses produksi",
    "Kesalahan input bahan baku",
    "Gangguan sistem operasional",
    "Penurunan kualitas produk"
  ];

  data.forEach((post, i) => {
    const status = getStatus(post.id);

    const judul = daftarMasalah[post.id % daftarMasalah.length];
    const deskripsi = `Insiden berupa ${judul.toLowerCase()} sedang dalam
    proses penanganan oleh tim produksi. (ID: ${post.id})`;

    cInsiden.innerHTML += `
```

```

<div class="col-md-6 col-lg-4">
  <div class="card-insiden">
    <div class="insiden-meta">
      <span>TKT-${String(post.id).padStart(4,"0")}</span>
      <span class="${status.cls}">${status.label}</span>
    </div>

    <p class="insiden-title"><b>${judul}</b></p>
    <p class="insiden-desc">${deskripsi}</p>

    <div class="insiden-footer">
      <span>Pelapor: User #${post.userId}</span>
      <button class="btn-tindak" data-id="${post.id}">
        Tindak Lanjut
      </button>
    </div>
  </div>
</div>
</div>
`;
});
}

// Klik tindak lanjut
cInsiden.addEventListener("click", (e) => {
  if (e.target.classList.contains("btn-tindak")) {
    alert(`Insiden ${e.target.dataset.id} sedang diproses oleh tim
maintenance`);
  }
});

// ===== KARYAWAN =====
function renderKaryawan(data) {
  cKaryawan.innerHTML = "";

  document.getElementById("jumlahKaryawan").textContent =
    `${data.length} karyawan`;

  if (data.length === 0) {
    cKaryawan.innerHTML = `
    <div class="col-12 text-center">
      <p>Tidak ada data karyawan</p>
    </div>`;
    return;
  }

  data.forEach((user) => {

```

```

cKaryawan.innerHTML += `
  <div class="col-md-6 col-lg-4">
    <div class="card-karyawan">
      <div class="avatar">${inisial(user.name)}</div>
      <p><b>${user.name}</b></p>
      <p>${user.email}</p>
      <p>${user.phone}</p>
      <p>${user.address.city}</p>

      <button class="btn-detail"
        data-user="${encodeURIComponent(JSON.stringify(user))}">
        Lihat Detail
      </button>
    </div>
  </div>
`;
});
}

// Detail modal
cKaryawan.addEventListener("click", (e) => {
  if (!e.target.classList.contains("btn-detail")) return;

  const user = JSON.parse(
    decodeURIComponent(e.target.dataset.user)
  );

  document.getElementById("detailNama").textContent = user.name;
  document.getElementById("detailUsername").textContent =
user.username;
  document.getElementById("detailEmail").textContent = user.email;
  document.getElementById("detailTelepon").textContent = user.phone;
  document.getElementById("detailWebsite").textContent = user.website;
  document.getElementById("detailPerusahaan").textContent =
user.company?.name ?? "-";
  document.getElementById("detailAlamat").textContent =
`${user.address.street}, ${user.address.suite}, ${user.address.city}
${user.address.zipcode}`;

  modalDetailBS.show();
});

// ===== FETCH DATA =====
async function muatData() {
  setLoading(true);

```



```
try {
  const [resPosts, resUsers] = await Promise.all([
    fetch(API_POSTS),
    fetch(API_USERS)
  ]);

  if (!resPosts.ok) throw new Error("Gagal mengambil data insiden");
  if (!resUsers.ok) throw new Error("Gagal mengambil data karyawan");

  const posts = await resPosts.json();
  const users = await resUsers.json();

  semuaKaryawan = users;

  renderInsiden(posts);
  renderKaryawan(users);

  statusBadge.className = "status-badge status-online";
  statusBadge.innerHTML = `<span class="status-dot"></span> Online`;

} catch (err) {
  alert(err.message);
}

setLoading(false);
}

// ===== FILTER =====
btnFilter.addEventListener("click", () => {
  if (semuaKaryawan.length === 0) {
    alert("Muat data terlebih dahulu");
    return;
  }

  const keyword = prompt("Masukkan nama kota:").toLowerCase();
  if (!keyword) return;

  const hasil = semuaKaryawan.filter(u =>
    u.address.city.toLowerCase().includes(keyword)
  );

  renderKaryawan(hasil);
});

// ===== TAMBAH KARYAWAN =====
```

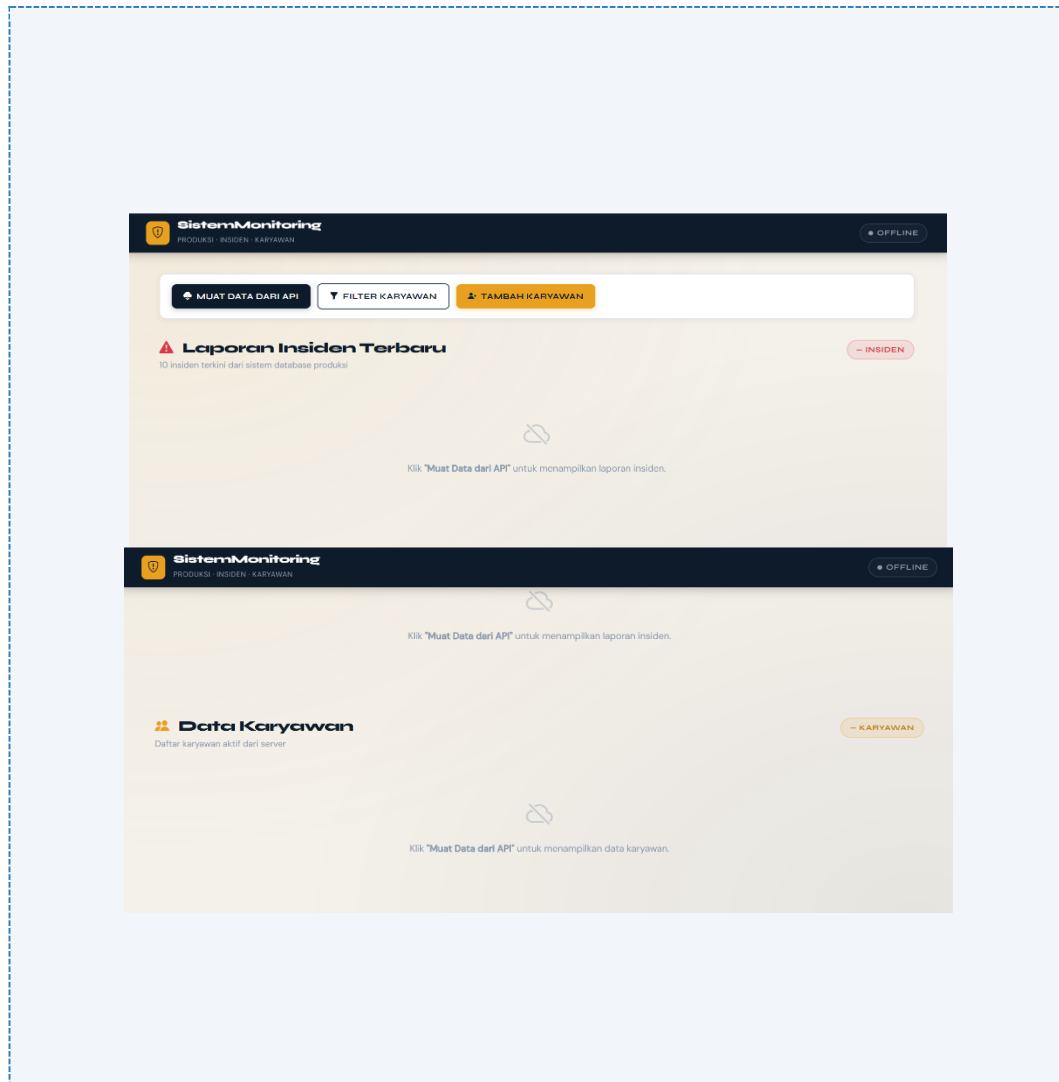
```
document.getElementById("formTambahKaryawan").addEventListener("submit", (e) => {
  e.preventDefault();

  const karyawanBaru = {
    id: semuaKaryawan.length + 1,
    name: document.getElementById("inputNama").value,
    email: document.getElementById("inputEmail").value,
    phone: "-",
    username:
document.getElementById("inputNama").value.toLowerCase().replace(/\s+/g,
"."),
    website: "-",
    company: { name: document.getElementById("inputPerusahaan").value },
    address: {
      street: "-",
      suite: "-",
      city: document.getElementById("inputKota").value,
      zipcode: "-"
    }
  };

  semuaKaryawan.push(karyawanBaru);
  renderKaryawan(semuaKaryawan);

  bootstrap.Modal.getInstance
(document.getElementById("modalTambah"))?.hide();
  e.target.reset();
});

// ===== LOAD =====
btnLoad.addEventListener("click", muatData);
```



Gambar F.3 – Hasil pembuatan mini proyek integrasi data produksi

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil program yang diperoleh menunjukkan kesesuaian yang baik dengan teori dasar Asynchronous JavaScript dalam proses komunikasi data dari sumber eksternal. Secara teoritis, pemrograman asinkron memungkinkan suatu proses dijalankan di latar belakang sehingga antarmuka aplikasi tetap responsif dan tidak mengalami pembekuan saat menunggu respons dari server. Pada program Laporan Insiden, konsep tersebut diterapkan melalui penggunaan Fetch API untuk mengambil data dari REST API publik. Implementasi ini terbukti mampu mendukung pertukaran data secara efisien, yang ditunjukkan oleh tampilan aplikasi yang tetap dapat digunakan dengan baik serta adanya indikator loading sebagai umpan balik visual selama proses pengambilan data berlangsung.

Kesesuaian dengan teori juga tampak pada penerapan Promise, pengolahan data berformat JavaScript Object Notation (JSON), serta

mekanisme penanganan kesalahan. Dalam pengembangan web modern, data yang diterima dari server perlu diuraikan terlebih dahulu agar dapat diproses oleh aplikasi di sisi klien. Sejalan dengan konsep tersebut, program ini melakukan parsing terhadap respons API menjadi objek JSON sebelum data dimanipulasi dan ditampilkan pada antarmuka. Selain itu, prinsip ketahanan sistem diwujudkan melalui penerapan error handling. Ketika terjadi kegagalan saat melakukan request ke API, aplikasi tidak berhenti secara tiba-tiba, tetapi mampu menangani kondisi tersebut dengan menampilkan pesan peringatan yang informatif kepada pengguna.

Penggabungan antara konsep asinkronus dan manipulasi DOM dalam program ini juga berhasil merepresentasikan teori integrasi aplikasi front-end dengan sistem data yang lebih luas. Secara konseptual, pendekatan tersebut memungkinkan aplikasi web berfungsi sebagai dasbor interaktif yang menghubungkan pengguna dengan sumber data terpusat. Dalam implementasinya, data yang diperoleh kemudian disesuaikan menjadi simulasi laporan insiden industri sehingga lebih mendekati kondisi nyata di lapangan. Hal ini sesuai dengan kebutuhan sistem operasional industri modern, di mana antarmuka pengguna harus mampu terhubung secara dinamis dengan database ERP maupun jaringan sensor IoT untuk mendukung proses maintenance yang cepat, efisien, dan mudah dipantau.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Proses pengambilan data dari REST API membutuhkan waktu sehingga pengguna harus menunggu respons dari server.	Menambahkan indikator loading agar pengguna mengetahui bahwa sistem sedang memproses permintaan data.
Koneksi internet yang tidak stabil dapat menyebabkan request API gagal dijalankan.	Menerapkan error handling menggunakan blok catch untuk menampilkan pesan peringatan tanpa membuat aplikasi mengalami crash.
Data yang diterima dari API masih berupa format mentah sehingga belum dapat langsung digunakan pada aplikasi.	Melakukan parsing JSON untuk mengubah respons API menjadi objek JavaScript yang mudah diolah dan ditampilkan.

G.3 Kaitan dengan Penerapan di Industri

Praktikum pada modul ini memiliki hubungan yang erat dengan penerapan sistem informasi di dunia industri, terutama dalam aspek komunikasi data terpusat dan pemantauan operasional secara real-time.

Penggunaan Asynchronous JavaScript melalui Fetch API menggambarkan mekanisme pemrosesan paralel yang banyak diterapkan pada antarmuka digital industri. Dalam kondisi operasional yang dinamis, kemampuan sistem untuk mengambil pembaruan data tanpa menghambat tampilan antarmuka sangat penting. Konsep asinkronus memungkinkan proses komunikasi dengan server berlangsung di latar belakang sehingga operator tetap dapat berinteraksi dengan dasbor tanpa terganggu oleh waktu tunggu selama proses pemuatan data.

Selain itu, integrasi REST API dan pengolahan format data pada program ini mencerminkan prinsip interoperabilitas yang terdapat dalam sistem Industrial Internet of Things (IIoT) maupun Enterprise Resource Planning (ERP). Data mentah yang diperoleh dari server eksternal diproses, dikonversi, dan disesuaikan menjadi laporan berbahasa Indonesia sehingga menyerupai proses standarisasi informasi dari berbagai sumber ke dalam format yang digunakan perusahaan. Penerapan error handling juga menunjukkan pentingnya keandalan sistem dalam menjaga kontinuitas operasional. Ketika terjadi gangguan jaringan atau kegagalan pengambilan data, sistem mampu memberikan peringatan yang jelas kepada pengguna sehingga risiko kesalahan pengambilan keputusan akibat terputusnya aliran informasi dapat diminimalkan.

H. KESIMPULAN

1	Penggunaan fungsi <code>fetch()</code> memungkinkan proses pengambilan data dari REST API dilakukan secara asinkron sehingga antarmuka pengguna tetap responsif dan tidak mengalami pembekuan selama menunggu respons dari server.
2	Metode <code>.json()</code> digunakan untuk mengubah respons mentah dari server menjadi objek JSON sehingga data dapat dibaca, diolah, dimanipulasi, dan ditampilkan sesuai kebutuhan aplikasi.
3	Manipulasi DOM secara dinamis dengan pemberian ID unik pada setiap elemen memungkinkan perubahan status visual dan pengaturan atribut dilakukan secara tepat pada elemen tertentu tanpa memengaruhi elemen lainnya.
4	Penerapan error handling melalui blok <code>.catch()</code> serta pengecekan <code>!response.ok</code> berfungsi menjaga kestabilan sistem, mencegah aplikasi mengalami crash, dan memastikan pengguna memperoleh informasi yang jelas ketika terjadi kegagalan koneksi.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://www.javascripttutorial.net/
[3]	https://www.javascripttutorial.net/javascript-dom/
[4]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[5]	https://www.w3schools.com/js/default.asp

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Saprina Melita Sari NIM:23051430012	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____